UML Diagrams: Class Diagrams, Relationships & Sequence Diagrams

1. Class Diagrams.

- A class diagram represents the structure of a system. It shows classes, their attributes, methods and relationships with other classes.

Class Diagram Structure :

A class is represented using a rectangle divided into three sections:

1. Class Name - Name of the class.
2. Attributes - Data or properties of the class.
3. Methods - Operations the class can perform.

Example:

```
+----------------------+
|        Car           |
+----------------------+
| brand: String        |
| model: String        |
| engine: int          |
+----------------------+
| startEngine()        |
| accelerate()         |
| brake()              |
| stopEngine()         |
+----------------------+
```

Access Modifiers

Symbol	Meaning
+	Public
-	Private
#	Protected

A class diagram can also represent abstract classes and generic classes, as shown in the lecture notes.

2. Relationships Between Classes

Relationships describe how classes are connected to or depend on each other.

The main relationships covered in the notes are:

- Association
- Inheritance
- Composition
- Aggregation

2.1. Association

Association represents a relationship between two classes.

Example: A person has a relationship with a house.

Person -------- House

Both classes can exist independently.

2.2. Inheritance

Inheritance represents an "is-a" relationship. A child class inherits properties and methods from a parent class.

Example: A Dog is an Animal.



Java example:

```
class Animal {
   void eat() {
      System.out.println("Eating");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Barking");
   }
}
```

Here, Dog inherits the eat() method from Animal.

2.3. Composition

Composition represents a strong "has-a" relationship. One object contains or owns another object as part of its structure.

Example: A house contains rooms.

House ◆-------- Room

The filled diamond represents composition in a UML class diagram.

Composition generally means the contained object's lifecycle is closely tied to its owning object.

2.4. Aggregation

Aggregation represents a weaker "has-a" relationship. One object groups or uses another object, but the contained object can exist independently.

Example: A team has players.

Team ◇-------- Player

The hollow diamond represents aggregation in a UML class diagram.

Quick Comparison

Relationship	Meaning	Example
Association	General relationship	Person and House
Inheritance	Is-a	Dog is an Animal
Composition	Strong has-a	House and Room
Aggregation	Weak has-a	Team and Player

Remember: Inheritance describes what something *is*. Composition and aggregation describe what something *has*.

3. Composition in Code

Composition can be represented by creating an object of one class inside another class.

```
class Engine {
   void start() {
      System.out.println("Engine started");
   }               }
```

```
class Car {
    private Engine engine = new Engine();
    void startCar() {
        engine.start();
    }
}
```

Here, Car contains an Engine object and uses it through startCar().

Key idea: An object can use another object's methods instead of implementing every operation itself.

4. Sequence Diagrams

- A sequence diagram represents the interactions between objects in the order they occur over time.
- While a class diagram shows the structure of a system, a sequence diagram shows how objects communicate to perform an operation.

Main Components

1. Objects - Represent the participants in an interaction.

2. Lifeline - A vertical dashed line showing an object's existence during the interaction.

3. Activation Bar - A narrow rectangle on a lifeline indicating when an object is executing an operation.

4. Messages - Arrows between lifelines representing communication between objects.

Types of Messages

- Synchronous message: The sender waits for the operation to complete before continuing.
- Asynchronous message: The sender can continue without waiting for the receiver to finish.
- Create message: Represents the creation of an object.
- Destroy message: Represents the end of an object's lifecycle.

5. Sequence Diagram Control Structures

Sequence diagrams can represent conditional and repeated operations.

- Alternative (alt) - Represents a choice between alternative paths, such as an if-else condition.
- Option (opt) - Represents an optional interaction, similar to an if condition without a required alternative.
- Loop (loop) - Represents a repeated interaction, such as a for or while loop.

6. Example: ATM Withdrawal Flow

The lecture notes use an ATM withdrawal to demonstrate how objects interact through a sequence diagram.

Participants

- User
- ATM
- Transaction
- Account
- Cash Dispenser

Flow

- The user initiates a withdrawal by providing the amount and account details.
- The ATM verifies the PIN.
- The account is verified.
- The transaction checks the requested amount.
- The transaction communicates with the cash dispenser.
- The cash dispenser dispenses the money when the transaction proceeds successfully.

Simplified Interaction

User

|

v

ATM

|

| Verify PIN

v

Transaction

|

| Verify account

v

Account

|

| Check withdrawal amount

v

Transaction

|

| Request cash

v

Cash Dispenser

|

| Dispense money

v

User

This is a simplified representation of the flow in the notes, not a complete production ATM design.

Key idea: A sequence diagram helps us understand which object calls which operation and in what order.

7. Class Diagram vs Sequence Diagram

Class Diagram	Sequence Diagram
Shows system structure	Shows object interactions
Represents classes and relationships	Represents messages exchanged between objects
Focuses on what exists	Focuses on what happens over time
Useful for designing class structure	Useful for understanding a particular workflow

8. Quick Revision

- Class diagram: Structure of the system.
- Association: General relationship between classes.
- Inheritance: Is-a relationship.
- Composition: Strong has-a relationship.
- Aggregation: Weak has-a relationship.
- Sequence diagram: Ordered communication between objects.
- Lifeline: Object's presence over time.
- Activation bar: Period when an object executes an operation.
- alt: Alternative paths.
- opt: Optional interaction.
- loop: Repeated interaction.